

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Desarrollo de Aplicaciones Web Client and Backend Development Frameworks

Integración de Swagger/OpenAPI en una API Spring Boot

Profesor: M. en C. José Asunción Enríquez Zárate

*Alumno: Estefany Karina Sánchez Calderón
estefanysanchez1495@gmail.com
7CM3*

24 de junio de 2026

Índice

1. Introducción	1
2. Conceptos	1
2.1. Swagger	1
2.2. OpenAPI	1
2.3. Springdoc OpenAPI	1
2.4. Spring Boot	1
2.5. Spring Security y JWT	1
3. Descripción del sistema	1
3.1. Módulos principales	1
3.2. Modelo de base de datos	2
4. Desarrollo	2
4.1. Dependencia de Springdoc OpenAPI	2
4.2. Configuración general de OpenAPI	2
4.3. Acceso a Swagger UI	3
4.4. Documentación de controladores	3
4.5. Endpoints documentados	3
4.6. Implementación de Spring Security	4
5. Resultados	4
5.1. Pruebas sugeridas desde Swagger UI	6
5.2. Observación sobre pruebas automáticas	7
6. Lista de capturas pendientes	7
7. Conclusiones	8
8. Referencias Bibliográficas	8

1 Introducción

La documentación de servicios REST es una parte importante del mantenimiento y consumo de una aplicación backend. Cuando un sistema cuenta con varios controladores, parámetros, códigos de respuesta y reglas de seguridad, resulta necesario contar con una interfaz que permita consultar la estructura de la API sin revisar directamente el código fuente.

En esta práctica se integró Swagger/OpenAPI en el backend del sistema *Codffee*, una API REST desarrollada con Spring Boot para la administración de una cafetería. La integración se realizó mediante Springdoc OpenAPI, lo cual permitió generar documentación automática, exponer la especificación OpenAPI en formato JSON y habilitar una interfaz interactiva mediante Swagger UI.

El objetivo principal fue documentar los endpoints del sistema, incluyendo operaciones, parámetros de entrada, códigos de respuesta, módulos principales y configuración de seguridad basada en JWT.

2 Conceptos

2.1 Swagger

Swagger es un conjunto de herramientas para describir, visualizar y probar APIs REST. En esta práctica se utilizó Swagger UI, una interfaz web que permite consultar los endpoints disponibles y ejecutar peticiones HTTP desde el navegador.

2.2 OpenAPI

OpenAPI es una especificación estándar para documentar APIs REST. Describe rutas, métodos HTTP, parámetros, cuerpos de solicitud, respuestas y mecanismos de autenticación. En el proyecto Codffee se genera automáticamente desde las anotaciones del backend.

2.3 Springdoc OpenAPI

Springdoc OpenAPI es una librería que integra OpenAPI con aplicaciones Spring Boot. Su dependencia principal permite generar la ruta `/v3/api-docs` y la interfaz `/swagger-ui/index.html`.

2.4 Spring Boot

Spring Boot es un framework de Java que simplifica el desarrollo de aplicaciones backend. El proyecto Codffee utiliza Spring Boot para exponer controladores REST, administrar seguridad, conectarse a MySQL y organizar la lógica por servicios, repositorios y entidades.

2.5 Spring Security y JWT

Spring Security se usa para proteger los endpoints de la aplicación. En Codffee se implementó autenticación mediante JWT. Los usuarios inician sesión, reciben un token y posteriormente lo envían como encabezado `Authorization: Bearer <token>` para consumir rutas protegidas.

3 Descripción del sistema

Codffee es un sistema backend para una cafetería. La API permite administrar usuarios, autenticación, productos, categorías, pedidos, detalles de pedidos, reportes PDF y envío de correos de prueba.

3.1 Módulos principales

- **Autenticación:** registro e inicio de sesión con JWT.
- **Usuarios:** administración de cuentas, roles y estado activo.
- **Perfil:** consulta y actualización de datos del usuario autenticado.
- **Categorías:** administración de categorías del catálogo.
- **Productos:** gestión del inventario y productos disponibles.
- **Pedidos:** creación, consulta, detalle, cancelación y cambio de estado.

- **Reportes:** generación de reportes PDF.
- **Correos:** prueba de configuración SMTP.

3.2 Modelo de base de datos

Las entidades principales del proyecto se resumen en la siguiente tabla:

Entidad	Descripción
Usuario	Almacena nombre, correo, contraseña, rol, estado activo y fecha de registro.
Categoría	Representa las categorías del catálogo de productos.
Producto	Contiene nombre, descripción, precio, stock, URL de imagen, disponibilidad y categoría.
Pedido	Registra usuario, fecha, estado, total, método de pago y observaciones.
DetallePedido	Relaciona un pedido con productos, cantidades, precio unitario y subtotal.

4 Desarrollo

4.1 Dependencia de Springdoc OpenAPI

Para integrar Swagger/OpenAPI se utilizó la dependencia `springdoc-openapi-starter-webmvc-ui`. Esta dependencia habilita tanto la especificación OpenAPI como la interfaz Swagger UI.

Código 1: Dependencia de Springdoc OpenAPI en pom.xml.

```

1 <dependency>
2   <groupId>org.springdoc</groupId>
3   <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
4   <version>3.0.3</version>
5 </dependency>

```

También se ajustó la versión de Java del proyecto a 21, debido a que el equipo local cuenta con JDK 21 instalado.

Código 2: Configuración de Java en pom.xml.

```

1 <properties>
2   <java.version>21</java.version>
3 </properties>

```

4.2 Configuración general de OpenAPI

Se configuró una clase `OpenApiConfig` para definir los datos principales de la documentación y el esquema de seguridad basado en JWT.

Código 3: Configuración principal de OpenAPI.

```

1 @Configuration
2 public class OpenApiConfig {
3
4     @Bean
5     public OpenAPI codffeeOpenAPI() {
6         final String securitySchemeName = "bearerAuth";
7
8         return new OpenAPI()
9             .info(new Info()
10                 .title("Codffee API")
11                 .version("1.0")
12                 .description("API REST para el sistema Codffee"))
13             .addServersItem(new Server()
14                 .url("http://localhost:8080")
15                 .description("Ambiente local"))

```

```

16         .addSecurityItem(new SecurityRequirement().addList(securitySchemeName
17     ))
18     .components(new Components()
19         .addSecuritySchemes(securitySchemeName,
20             new SecurityScheme()
21                 .name(securitySchemeName)
22                 .type(SecurityScheme.Type.HTTP)
23                 .scheme("bearer")
24                 .bearerFormat("JWT")));
25 }

```

4.3 Acceso a Swagger UI

En la configuración de seguridad se permitió el acceso público a Swagger UI y a la especificación OpenAPI. Esto es necesario para que la documentación pueda consultarse sin iniciar sesión.

Código 4: Rutas públicas para Swagger y OpenAPI en SecurityConfig.

```

1 .authorizeHttpRequests(auth -> auth
2     .requestMatchers(
3         "/swagger-ui/**",
4         "/v3/api-docs/**",
5         "/api/auth/**",
6         "/api/correos/prueba")
7     .permitAll()
8     .anyRequest().authenticated())

```

4.4 Documentación de controladores

Los controladores fueron documentados con anotaciones de OpenAPI. Se agregaron etiquetas por módulo con `@Tag`, descripciones por operación con `@Operation`, parámetros con `@Parameter` y respuestas esperadas con `@ApiResponse`.

Código 5: Ejemplo de documentación en un controlador REST.

```

1 @RestController
2 @RequestMapping("/api/productos")
3 @Tag(name = "Productos", description = "Gestion del catalogo de productos de la
4     cafeteria")
5 public class ProductoController {
6     @GetMapping("/{id}")
7     @Operation(summary = "Buscar producto por ID",
8         description = "Consulta el detalle de un producto del catalogo.")
9     @ApiResponses({
10         @ApiResponse(responseCode = "200", description = "Producto encontrado"),
11         @ApiResponse(responseCode = "404", description = "Producto no encontrado")
12     })
13     public Producto buscarPorId(
14         @Parameter(description = "ID del producto", example = "1")
15         @PathVariable Long id) {
16         return productoService.buscarPorId(id);
17     }
18 }

```

4.5 Endpoints documentados

La siguiente tabla resume los módulos documentados en Swagger UI:

Módulo	Endpoints principales	Seguridad
Autenticación	POST /api/auth/login, POST /api/auth/register	Público

Productos	GET /api/productos, GET /api/productos/disponibles, POST /api/productos, PUT /api/productos/{id}	Mixto
Categorías	GET /api/categorias, GET /api/categorias/activas, POST /api/categorias	Mixto
Usuarios	GET /api/usuarios, POST /api/usuarios, PUT /api/usuarios/{id}	ADMIN
Perfil	GET /api/perfil, PUT /api/perfil	JWT
Pedidos	GET /api/pedidos, POST /api/pedidos, PUT /api/pedidos/{id}/cancelar	JWT
Reportes	GET /api/reportes/pedidos/pdf, GET /api/reportes/pedidos/pdf/filtrado	ADMIN
Correos	POST /api/correos/prueba	Público

4.6 Implementación de Spring Security

La seguridad del sistema funciona con sesiones sin estado mediante `SessionCreationPolicy.STATELESS`. El filtro `JwtAuthenticationFilter` se ejecuta antes de `UsernamePasswordAuthenticationFilter`, por lo que las solicitudes protegidas se validan con el token JWT enviado por el cliente.

Código 6: Registro del filtro JWT en la cadena de seguridad.

```

1 .sessionManagement(session ->
2     session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
3 .addFilterBefore(jwtAuthenticationFilter,
4     UsernamePasswordAuthenticationFilter.class);

```

Las reglas de autorización principales son:

- **Público:** login, registro, Swagger UI, OpenAPI JSON, productos disponibles y categorías activas.
- **ADMIN:** administración de usuarios y reportes.
- **ADMIN o PERSONAL:** administración de productos y categorías.
- **ADMIN, PERSONAL o CLIENTE:** operaciones relacionadas con pedidos.
- **Autenticado:** consulta y actualización de perfil.

5 Resultados

El proyecto compila correctamente al omitir pruebas que dependen de la conexión local a MySQL. La compilación se verificó con Maven mediante el siguiente comando:

Código 7: Comando de compilación utilizado.

```

1 ./mvnw -DskipTests package

```

El resultado obtenido fue **BUILD SUCCESS**, lo cual confirma que la integración de Swagger/OpenAPI no genera errores de compilación.

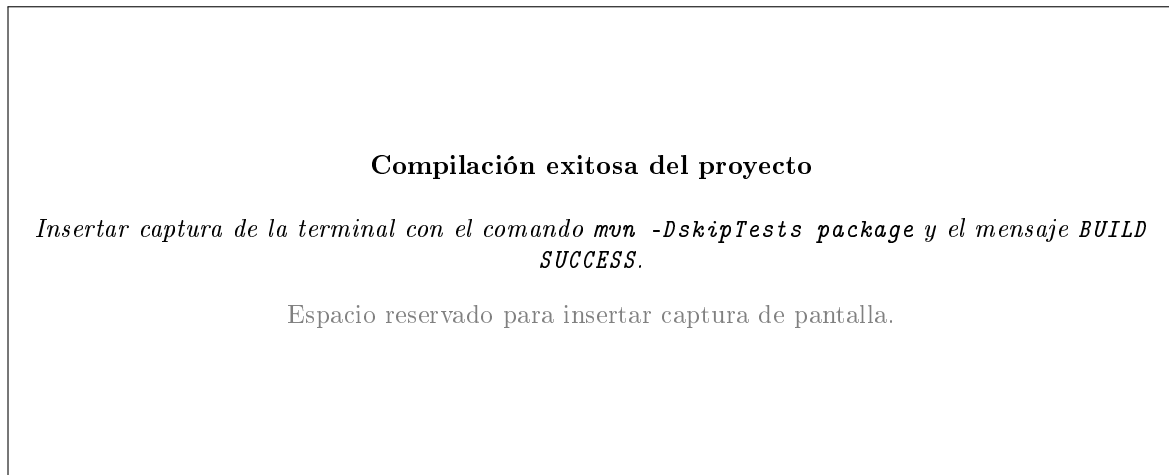


Figura 1: Compilación exitosa del proyecto

La interfaz Swagger UI debe estar disponible en la siguiente ruta cuando el backend se encuentre ejecutándose:

<http://localhost:8080/swagger-ui/index.html>

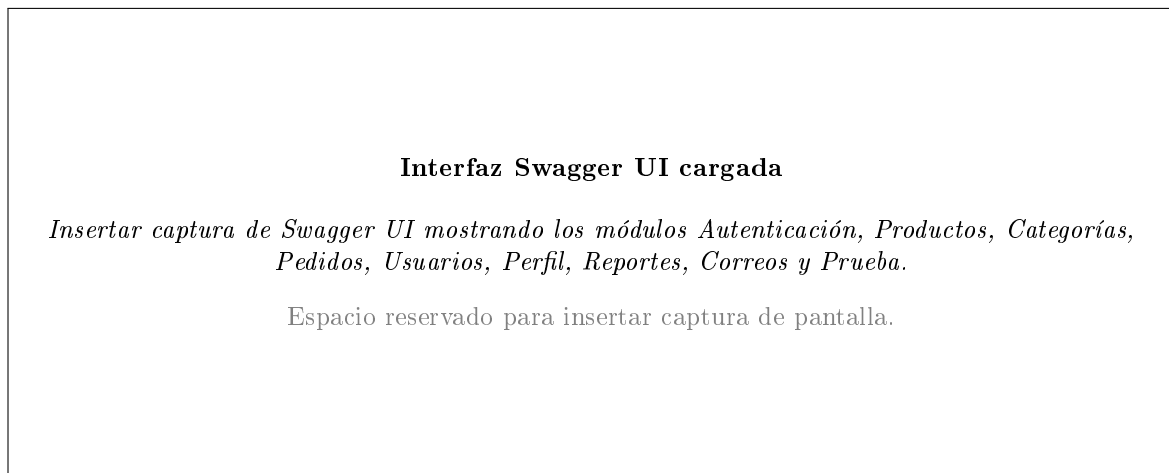


Figura 2: Interfaz Swagger UI cargada

La especificación OpenAPI en formato JSON se consulta en:

<http://localhost:8080/v3/api-docs>

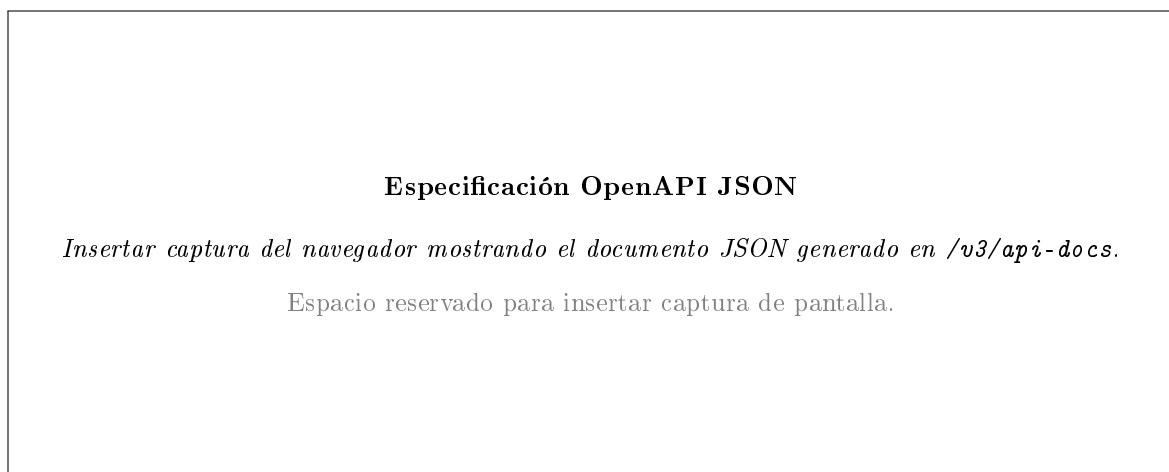


Figura 3: Especificación OpenAPI JSON

5.1 Pruebas sugeridas desde Swagger UI

Para validar el funcionamiento desde Swagger UI se recomienda realizar las siguientes pruebas:

Prueba	Endpoint	Resultado esperado
Probar disponibilidad del backend	GET /api/prueba	Respuesta HTTP 200 con mensaje de funcionamiento.
Iniciar sesión	POST /api/auth/login	Respuesta con token JWT tipo Bearer.
Listar productos disponibles	GET /api/productos/disponibles	Lista de productos disponibles.
Listar categorías activas	GET /api/categorias/activas	Lista de categorías activas.
Consultar perfil autenticado	GET /api/perfil	Datos del usuario autenticado al enviar JWT.
Generar reporte PDF	GET /api/reportes/pedidos/pdf	Archivo PDF como respuesta.

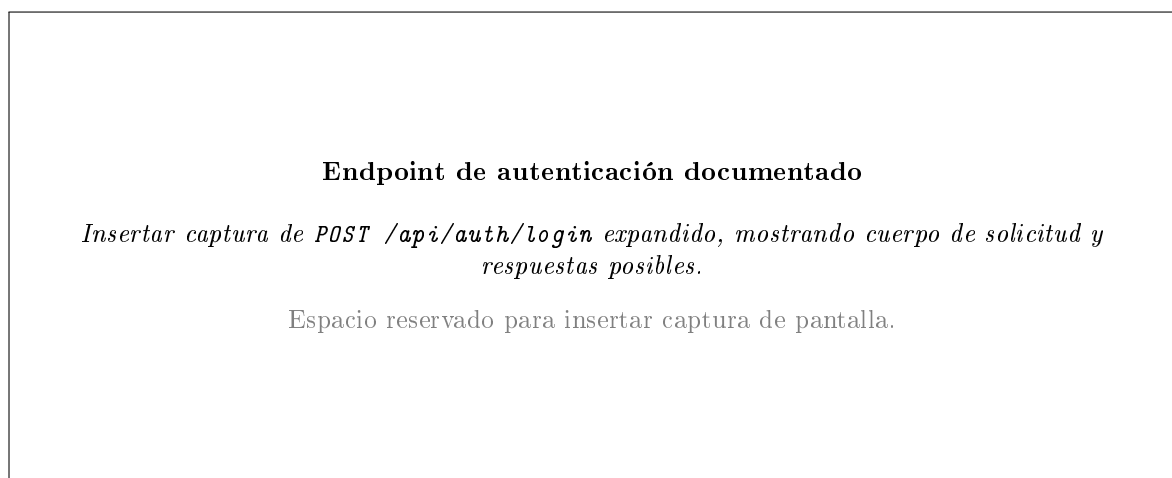


Figura 4: Endpoint de autenticación documentado

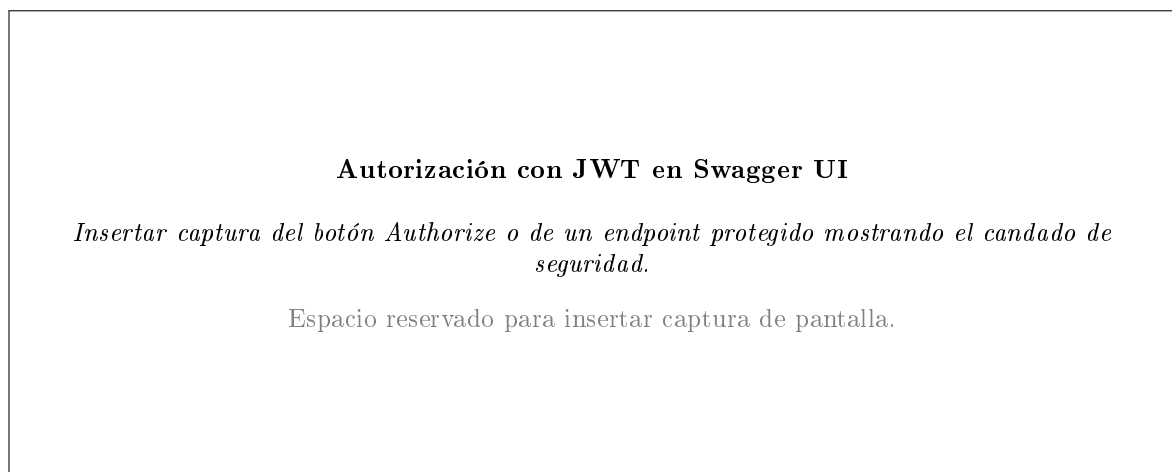


Figura 5: Autorización con JWT en Swagger UI

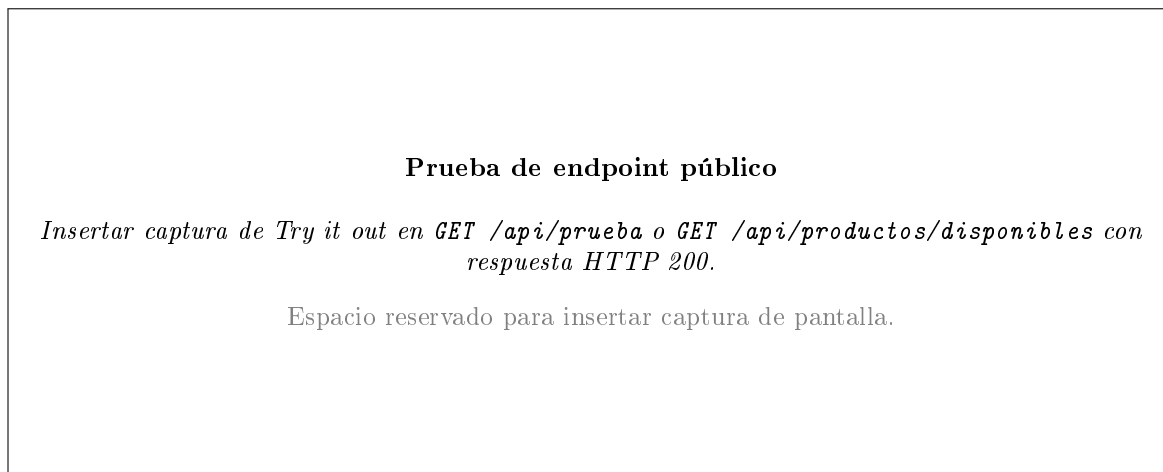


Figura 6: Prueba de endpoint público

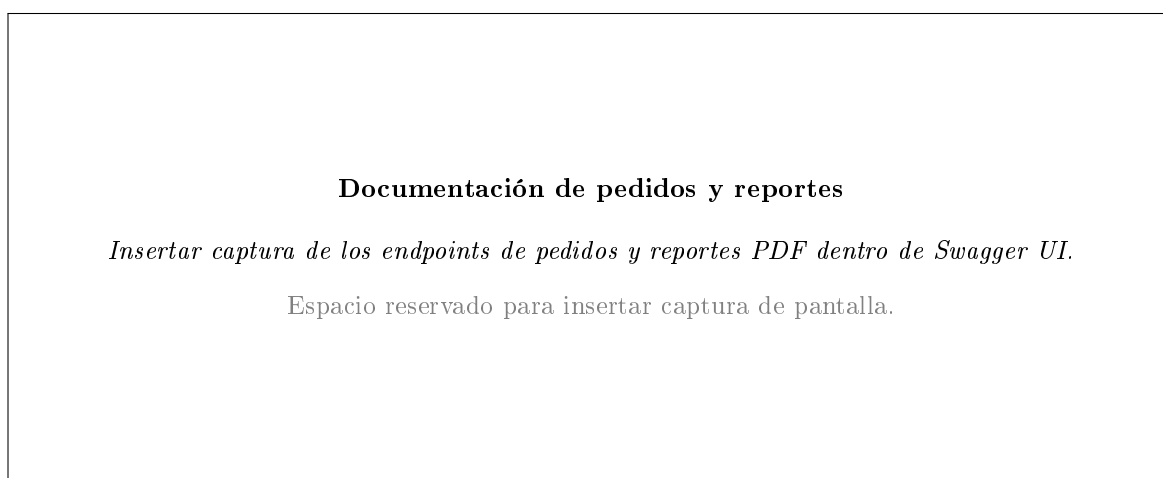


Figura 7: Documentación de pedidos y reportes

5.2 Observación sobre pruebas automáticas

Al ejecutar `mvn test`, el proyecto compila, pero la prueba de carga de contexto falla si MySQL no se encuentra en ejecución. El error corresponde a una falla de conexión con la base de datos local, no a un problema de Swagger/OpenAPI. Para ejecutar las pruebas completas se debe iniciar MySQL o levantar los servicios definidos en `docker-compose.yml`.

Código 8: Comando sugerido para iniciar servicios con Docker.

```
1 docker compose up --build -d
```

6 Lista de capturas pendientes

1. Terminal con `mvn -DskipTests package` y resultado `BUILD SUCCESS`.
2. Docker Desktop iniciado o backend ejecutándose localmente.
3. Swagger UI en <http://localhost:8080/swagger-ui/index.html>.
4. Especificación OpenAPI JSON en <http://localhost:8080/v3/api-docs>.
5. Endpoint `POST /api/auth/login` expandido.
6. Botón *Authorize* con token Bearer JWT.
7. `GET /api/productos/disponibles` probado desde Swagger UI.

8. GET `/api/categorias/activas` probado desde Swagger UI.
9. Endpoints de pedidos documentados.
10. Endpoint de reportes PDF mostrando respuesta `application/pdf`.

7 Conclusiones

La integración de Swagger/OpenAPI permitió generar documentación automática para el backend Codffee. Con esta herramienta, los endpoints pueden consultarse desde una interfaz interactiva, lo que facilita su prueba, validación y comprensión por parte de otros desarrolladores.

La práctica también permitió reforzar la importancia de documentar parámetros, respuestas y reglas de seguridad. Al incluir el esquema `bearerAuth`, Swagger UI puede representar correctamente los endpoints protegidos por JWT y permitir pruebas autenticadas desde el navegador.

Finalmente, el proyecto quedó preparado para documentar los módulos principales solicitados: productos, categorías, pedidos, autenticación, seguridad y reportes. La compilación del backend fue exitosa, quedando pendiente únicamente levantar MySQL o Docker Desktop para tomar las capturas finales de funcionamiento.

8 Referencias Bibliográficas

Referencias

- [1] Springdoc OpenAPI, *Springdoc OpenAPI Documentation*. <https://springdoc.org/>
- [2] Swagger, *API Documentation and Design Tools for Teams*. <https://swagger.io/tools/swagger-ui/>
- [3] OpenAPI Initiative, *OpenAPI Specification*. <https://spec.openapis.org/oas/latest.html>
- [4] Spring, *Spring Security Reference*. <https://docs.spring.io/spring-security/reference/>
- [5] Spring, *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>